



Sandya Mannarswamy

In this month's column, we will discuss a few more interesting issues related to multi-threaded programming, such as scalability and lock contention.

Many readers sent in answers to last month's takeaway question, but none of them were absolutely correct. So I'm going to keep this question open for this month's column as well. Here is a clue, though. Assume that you are solving it on a hardware system where $X\%Y$ is evaluated as $X - (X/Y)*Y$. Think whether this entire operation is atomic.

There were a few reader requests to discuss the scalability of multi-threaded code and lock contention, so we take a look at these topics in this month's column.

Why doesn't my application scale?

Many a programmer has ported a sequential application to a multi-threaded application, and then found that he is not getting the expected increase in performance. If the performance of the sequential application was X , when parallelising it and running it with, say, four threads, the programmer expects a $4X$ performance. However, this never happens in reality; do you know why?

When you parallelise a sequential application, not all parts of the application are parallelised. Only the part of the application which gets parallelised can be speeded up by using more processors. Therefore, the amount of speed-up you can get depends on how much of your original serial application is amenable to being parallelised. This principle is the well-known Amdahl's Law.

Amdahl's Law captures the relationship between the expected speed-up after parallelising a part of the sequential program, relative to the time taken to execute the program serially. Consider a sequential program where a proportion P of the original serial program has now been parallelised, such that the parallelised part has a speed-up of S . Amdahl's Law states that the maximum overall speed-up of the parallelised version is given by $1/((1-P) + P/S)$. If we assume that the running time of the original sequential program was 1 unit of time, then we can specify the speed up as $1/\text{running time of the parallelised version}$. In the parallelised version, the improved/parallel part now takes only P/S units of time to execute, while the non-parallelised part takes $(1-P)$ units of time to execute. Hence, the total running time of the parallelised version is $P/S + (1-P)$. Therefore, the expected speed up compared to the sequential program is $1/((1-P) + P/S)$.

Let us assume that I can parallelise the entire sequential application. Then, assuming I have four processors on which I can run the application, compared to the single processor on which I was running the serial application, can I get $4X$ speed up? Let us consider the following code snippet, in which we have an array of integers, which are summed up, and the sum is printed in a sequential fashion.

```
int SumArray(int Array[], int num_elems)
{
```

```
    int Sum = 0;

    for (int i = 0; i < num_elems; i++)
        Sum += Array[i];

    return Sum;
}
```

A trivial parallelisation for this function would be to sum different chunks of the array in different threads, as shown in this modified snippet:

```
#include <pthread.h>

pthread_mutex_t sum_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_t* Threads[MAX_THREADS];
int Array[NUM_ELEMS];
int Sum;

void SumArray(int chunk_size)
{
    long tid = pthread_self();
    int start = tid*chunk_size;
    int stop = tid*chunk_size + chunk_size;

    for (int i = start; i < stop; i++)
    {
        pthread_mutex_lock(&sum_mutex);
        Sum += Array[i];
        pthread_mutex_unlock(&sum_mutex);
    }
}

int foo(int num_threads)
{
    for (int i = 0; i < num_threads; i++)
        pthread_create(&Threads[i], NULL, SumArray, chunk_sz);
    printf("Final Sum is %d\n", Sum);
}
```

In the above code snippet, each thread operates on an independent section of the array. Therefore, our first expectation is that this function should scale linearly with the increase in number of processors. However, it is not so. Why? Each thread operates on an independent array section, but they all update the common shared global variable, *Sum*, which becomes the hotspot. Since this variable can be updated by multiple threads, the programmer has protected